

Plan de Trabajo

Proyecto 2 — Sistema Multimodal de Recuperación y Búsqueda

Curso: Base de Datos II (CS2042)

Profesor: Heider Sanchez

Equipo: 4 integrantes

Repositorio: <https://github.com/AlejandroMarceloCh/PROYECTOBD2-parte2>

Dataset: Free Music Archive (FMA) — <https://github.com/mdeff/fma>

Material de referencia: Enunciado del proyecto; material del curso: “Técnicas de Indexación”, “Algoritmos Externos”, “Recuperación de Información Textual”, “Índices Avanzados en PostgreSQL” y “BD Vectoriales — Modelos de Recuperación Multimedia”.

1. De qué trata el proyecto

Construimos un motor de recuperación que funciona igual para tres tipos de dato: texto, audio e imagen. La idea de fondo es una sola: cualquier objeto —una canción, una ficha de metadata, una portada— se convierte en un histograma de “palabras” y se indexa con un índice invertido propio. Las palabras cambian según la modalidad (términos para texto, palabras acústicas para audio, palabras visuales para imagen), pero la tubería es la misma:

split → extractor de características → codebook → índice invertido

Sobre ese motor montamos dos aplicaciones de búsqueda y, para sustentar las decisiones de diseño, comparamos nuestro índice contra los índices nativos de PostgreSQL.

Trabajamos sobre FMA porque es el único dataset del enunciado con audio crudo real (mp3): eso nos deja extraer MFCC propios en vez de usar características precalculadas. De los ~25 000 tracks de la versión **medium** sacamos cuatro segmentos por track, lo que nos lleva a las 100 000 unidades indexables que pide la evaluación.

2. Objetivos

1. Implementar la arquitectura unificada *split → extractor → codebook → índice invertido*, agnóstica a la modalidad.
2. Persistir codebook, histogramas y metadatos en PostgreSQL.
3. Comparar el índice propio contra los baselines nativos que pide el enunciado: GIN/GiST para texto y pgvector (HNSW/IVF) para audio e imagen.
4. Evaluar el sistema a 1K, 10K y 100K chunks midiendo latencia, throughput, precisión/recall, memoria e I/O.
5. Entregar dos aplicaciones funcionando en una demo en vivo.

3. Especificaciones del entregable

Lo que el profesor pide, sin interpretación:

- Repositorio en GitHub documentado, con README/Wiki como informe.
- `docker-compose` que levante PostgreSQL con la extensión pgvector.
- Índice invertido propio comparado contra GIN/GiST (texto) y pgvector (vectorial).
- Evaluación a 1K / 10K / 100K chunks con gráficos.
- Al menos dos aplicaciones en demo viva.
- Tablero en GitHub Projects con los issues cerrados desde los commits.
- Presentación oral.

El profesor evalúa la cantidad y el reparto de commits. Por eso trabajamos con commits chicos y diarios, cada integrante en su carpeta para no pisarse, y cerramos cada issue desde el commit correspondiente.

4. Arquitectura del sistema

El núcleo define cuatro interfaces y nadie las toca una vez fijadas. Cada modalidad las implementa por su lado:

- **Splitter** — parte el objeto en unidades (chunks).
- **FeatureExtractor** — saca características de cada chunk (TF-IDF, MFCC o SIFT).
- **Codebook** — cuantiza las características contra un vocabulario y arma el histograma.
- **InvertedIndex** — guarda y consulta los histogramas.

Definición de chunk por modalidad (fijada en Fase 0, no se cambia sin acuerdo del equipo):

- **Audio:** un segmento de ~ 7.5 s. Un frame MFCC es una característica, no un chunk. $25K \text{ tracks} \times 4 \text{ segmentos} = 100K \text{ chunks}$.
- **Texto:** un documento de metadata por track.
- **Imagen:** una portada de álbum.

La extensibilidad sale de esta separación: para una tercera aplicación basta con agregar una clase en `src/apps` sin tocar el motor.

5. División del equipo

Cada integrante es dueño de una modalidad o capa completa, alineada con lo que se ve en el curso. El reparto es por responsabilidad, no por horas: quien construye un pipeline lo lleva de punta a punta, incluida su comparativa contra el método nativo de PostgreSQL.

Antes de los briefs, una aclaración sobre las fuentes: el curso enseña los *conceptos* (índice invertido, TF-IDF, similitud de coseno, MFCC, SIFT, KNN, distancias). Las *herramientas* concretas de la comparativa —GIN, pgvector con HNSW/IVF— las exige el enunciado y no aparecen en los slides; se investigan aparte.

Integrante 1 — Recuperación textual

Responsabilidad: El pipeline de texto completo y su comparativa contra la búsqueda full-text de PostgreSQL.

Módulos: `src/text/`

Trabajo:

- Preprocesamiento: tokenización, normalización, eliminación de stopwords y stemming sobre la metadata de FMA (título, artista, género, tags y la biografía del artista).
- Vocabulario: construir el codebook textual con las palabras más frecuentes del corpus (top-k tras el preproceso).
- Ponderación TF-IDF de los histogramas y consulta por similitud de coseno: intersección de *posting lists* ordenadas por docID y ranking por coseno (acumulación de scores con los pesos IDF).
- Construcción del índice por bloques en disco y fusión *k-way* de las posting lists con un *min-heap* sobre los `term_id`. Este es el mecanismo de los algoritmos externos del curso; en recuperación de información se lo conoce como SPIMI/BSBI.
- La construcción se monta sobre la interfaz `InvertedIndex` que entrega el núcleo (Alejandro); el Integrante 1 aporta el preproceso, la ponderación y la construcción por bloques.
- Comparativa contra GIN y `to_tsvector/to_tsquery` sobre el mismo corpus.

Temas del curso que aplica: “Recuperación de Información Textual” (índice invertido, TF-IDF, similitud de coseno, intersección de posting lists); “Algoritmos Externos” (la fusión *k-way* con min-heap que sostiene la construcción del índice). La comparativa contra GIN/GiST la pide el enunciado: el material cubre el full-text con `to_tsvector` y el marco GiST, pero GIN como tal no está en los slides.

Integrante 2 — Recuperación de audio

Responsabilidad: El pipeline de audio completo y su comparativa contra pgvector.

Módulos: `src/audio/`

Trabajo:

- Segmentar cada track en ventanas de ~ 7.5 s y extraer los MFCC por ventana.
- Construir el codebook acústico con MiniBatchKMeans (el K-Means clásico no aguanta millones de frames).
- Cuantizar cada segmento contra el codebook y acumular su histograma de palabras acústicas.
- Búsqueda por similitud sobre los histogramas y comparativa contra pgvector con índices HNSW e IVF.
- Correr la extracción pesada (MFCC + clustering) en Google Colab y bajar solo los histogramas; el disco local no aguanta el dataset completo.
- Control de calidad previo: descartar tracks corruptos o silenciosos antes de extraer.

Temas del curso que aplica: “BD Vectoriales — Modelos de Recuperación Multimedia”. El profesor enseña este pipeline tal cual: *waveform* $\rightarrow$ ventana $\rightarrow$ MFCCs $\rightarrow$ cuantización $\rightarrow$ histograma; de ahí también salen la búsqueda KNN, las distancias y el trade-off precisión/recall. La comparativa contra pgvector (HNSW/IVF) es un baseline que pide el enunciado; pgvector no se cubre en los slides.

Integrante 3 — Recuperación de imágenes y capa de base de datos

Responsabilidad: El pipeline de imagen sobre las portadas y el esquema de PostgreSQL que sostiene a todo el equipo.

Módulos: `src/image/`, `src/db/`, `sql/`

Trabajo:

- SIFT: detección de keypoints y descriptores de 128 dimensiones (número variable por imagen).
- Codebook visual con K-Means y representación Bag of Visual Words: cada portada queda como un histograma de palabras visuales de dimensión fija.
- Control de calidad del set de portadas y documentar el límite de diseño: las portadas son por álbum (~ 15 K únicas), así que la búsqueda visual devuelve álbumes, no tracks.
- Esquema de la base: tablas de productos, codebook e histogramas; instalación de la extensión pgvector y migraciones.
- Configurar y parametrizar los índices nativos (GiST y pgvector) que usan las tres comparativas. Esto es infraestructura: deja los índices listos, pero cada dueño de modalidad corre su propio benchmark sobre ellos.

Temas del curso que aplica: “BD Vectoriales — Modelos de Recuperación Multimedia” (SIFT, palabras visuales, BoVW); “Índices Avanzados en PostgreSQL” (GiST, SP-GiST, BRIN) como contexto de los índices del motor. La comparativa contra pgvector la pide el enunciado; no está en los slides.

Alejandro Marcelo — Núcleo, backend, aplicaciones y evaluación

Responsabilidad: El motor común, las dos aplicaciones, el backend, el banco de pruebas y la coordinación del repositorio.

Módulos: `src/core/`, `src/apps/`, `backend/`, `eval/`, infraestructura (Docker) y el scraper de portadas.

Trabajo:

- Definir las interfaces ABC del núcleo (`Splitter`, `FeatureExtractor`, `Codebook`, `InvertedIndex`) y dejarlas estables para que las tres modalidades enganchen.
- Implementar el índice invertido propio y genérico en el núcleo: diccionario más *posting lists*, con intersección y ranking por coseno. Las tres modalidades lo reutilizan; cada pipeline aporta su construcción y su preproceso.
- App 1 — identificación de audio (tipo Shazam): se sube un fragmento, se extrae su histograma y se devuelven los tracks más parecidos.
- App 2 — búsqueda por texto: una consulta en lenguaje natural devuelve tracks por género y tags.
- Backend en FastAPI que expone las dos apps; demo de punta a punta.
- Banco de pruebas a 1K/10K/100K chunks con sus gráficos; `docker-compose` con PostgreSQL + pgvector; scraper de portadas.
- Coordinación: tablero de issues, reparto de commits y revisión de los *merges*.

Temas del curso que aplica: “Técnicas de Indexación” (el índice invertido se apoya en los índices densos/sparse y en la teoría del B+Tree); “Arquitectura del DBMS” (el flujo de una consulta — parser, optimizer, executor, buffer— para entender el motor sobre el que corre todo).

6. Comparativas y evaluación (transversal)

Cada modalidad enfrenta su índice propio contra el método nativo que le corresponde. No se mezclan: GIN no aplica a audio y pgvector no aplica a full-text.

Modalidad	Índice propio	Baseline nativo (enunciado)
Texto	Índice invertido + construcción <i>k-way</i>	GIN / GiST (full-text)
Audio	Histogramas acústicos	pgvector (HNSW / IVF)
Imagen	Histogramas visuales (BoVW)	pgvector (HNSW / IVF)

GIN/GiST y pgvector son los baselines que exige el enunciado; el curso cubre los conceptos (full-text con `to_tsvector`, el marco GiST, KNN y distancias sobre histogramas), no esas herramientas en sí.

Protocolo experimental (cerrado antes de empezar a medir):

- Conjunto de consultas fijo y separado (no visto durante la indexación).
- Mismo subconjunto, misma consulta y mismas condiciones para los tres métodos.
- Mismo k entre sistemas comparados.
- Se mide con caché fría y caché caliente, y se reportan las dos.
- Semillas fijas y documentadas.
- I/O en PostgreSQL con `EXPLAIN (ANALYZE, BUFFERS)`.

Para precisión/recall usamos como referencia el género: relevante = mismo género de primer nivel (FMA tiene 161 géneros jerárquicos en 16 raíces); como métrica secundaria, el subgénero o los tags.

7. Orden de trabajo

El reparto es por asignaciones, pero hay un orden lógico que respeta las dependencias entre módulos.

Fase 0 — Acuerdos de diseño (todos).

Fijar la definición de chunk por modalidad, el esquema de la base y el protocolo experimental. Aquí también se cierra el **contrato de entrega** entre los pipelines y el núcleo: el formato del histograma y de la metadata persistida. Nadie codea su pipeline antes de cerrar esto, o los módulos no encajan.

Fase 1 — Pipelines por modalidad (en paralelo).

Cada integrante construye su extractor, su codebook y sus histogramas sobre una muestra chica. Alejandro deja el núcleo y las interfaces listas al inicio de esta fase. Cada quien va dejando escrita la sección de su modalidad para el informe.

Fase 2 — Persistencia y comparativas.

Cargar los histogramas en PostgreSQL y levantar las comparativas contra los índices nativos.

Fase 3 — Aplicaciones y demo.

Backend, las dos apps y la demo de punta a punta. No arranca hasta que los tres pipelines estén persistidos (depende del contrato de la Fase 0).

Fase 4 — Evaluación e informe.

Correr el banco de pruebas a 1K/10K/100K y generar los gráficos. Cada integrante redacta la sección de su modalidad; Alejandro consolida arquitectura y evaluación en el README/Wiki.

Fase 5 — Presentación.

Cada integrante presenta su capa y Alejandro cubre arquitectura, apps y evaluación. Se preparan las diapositivas y se ensaya la demo de las dos apps.

8. Convenciones de trabajo

- Commits chicos y diarios, en español, descriptivos: `feat(audio): extracción MFCC por segmento`.

- Una tarea = un issue en GitHub Projects, cerrado desde el commit con `closes #N`.
- Una rama por feature, PR a `main` y revisión antes del merge.
- Cada integrante trabaja en su carpeta para no generar conflictos.